TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ

BỘ MÔN: CÔNG NGHỆ THÔNG TIN

BÀI TẬP LỚN

MÔN HỌC

TRÍ TUỆ NHÂN TẠO VÀ HỌC MÁY

ĐỀ TÀI : NHẬN DẠNG CỬ CHỈ TAY ĐIỀU KHIỂN TRÒ RẮN SĂN MỒI

Sinh viên	:	Lương Văn Học
Lớp	:	K58KTP
Giáo viên hướng dẫn	:	TS. Nguyễn Tuấn Linh

Thái Nguyên – 2026

TRƯỜNG ĐHKTCN	**CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM**
KHOA ĐIỆN TỬ	***Độc lập - Tự do - Hạnh phúc***

BÀI TẬP LỚN

MÔN HỌC: TRÍ TUỆ NHÂN TẠO VÀ HỌC MÁY

BỘ MÔN: CÔNG NGHỆ THÔNG TIN

Sinh viên: Lương Văn Học *MSSV:K225480106025*

Lớp: K58KTP *Ngành: Kỹ thuật phần mềm*

Giáo viên hướng dẫn: TS. Nguyễn Tuấn Linh

Ngày giao đề: 21/12/2025 *Ngày hoàn thành: 19/03/2026*

Tên đề tài:

Nhận dạng cử chỉ tay điều khiển trò rắn săn mồi

Yêu cầu:

Mô tả bài toán :

- Xây dựng hệ thống điều khiển trò chơi rắn săn mồi bằng cử chỉ tay.
- Sử dụng webcam để nhận hình ảnh bàn tay của người dùng.
- Nhận diện các cử chỉ tay tương ứng với các hướng điều khiển: lên, xuống, trái, phải.
- Chuyển kết quả nhận diện thành lệnh điều khiển trong trò chơi.

Mô tả dữ liệu :

- Dữ liệu được thu thập từ webcam trong quá trình người dùng thực hiện cử chỉ tay.
- Mỗi mẫu dữ liệu được lưu lại theo từng nhãn cử chỉ: lên, xuống, trái, phải.

- Dữ liệu được lưu dưới dạng các tệp .npy để phục vụ cho việc huấn luyện mô hình.

Mô hình học máy :

- Sử dụng thuật toán SVM để phân loại các cử chỉ tay.
- Dữ liệu thu thập được dùng để huấn luyện mô hình.
- Mô hình sau khi huấn luyện sẽ dự đoán cử chỉ tay theo thời gian thực.
- Kết quả dự đoán được sử dụng để điều khiển hướng di chuyển của rắn.

Giao diện :

- Gồm cửa sổ webcam để hiển thị và nhận diện cử chỉ tay.
- Gồm cửa sổ trò chơi rắn săn mồi hiển thị quá trình chơi.
- Hiển thị điểm số và thông báo kết thúc trò chơi.
- Có nút chơi lại để bắt đầu ván chơi mới.

GIÁO VIÊN HƯỚNG DẪN

(Kí và ghi rõ hộ tên)

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Thái Nguyên, ngày tháng năm 2026

GIÁO VIÊN HƯỚNG DẪN

(Ký ghi rõ họ tên)

MỤC LỤC

MỤC LỤC 1
LỜI CAM ĐOAN 3
DANH MỤC CÁC TỪ VIẾT TẮT 4
DANH MỤC HÌNH ẢNH 5
LỜI NÓI ĐẦU 6
LỜI CẢM ƠN 7
CHƯƠNG 1. TỔNG QUAN 8
1.1 Đặt vấn đề 8
1.2 Lý do chọn đề tài 8
1.3. Mục tiêu của đề tài 9
1.4. Đóng góp của báo cáo 9
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT & CÁC NGHIÊN CỨU LIÊN QUAN . 11
2.1. Các nghiên cứu liên quan 11
2.2. Cơ sở lý thuyết 11
CHƯƠNG 3. PHƯƠNG PHÁP THỰC HIỆN 13
3.1. Mô tả dữ liệu 13
3.2. Khám phá và Tiền xử lý dữ liệu 13
3.3. Mô hình đề xuất 14
CHƯƠNG 4: THỰC NGHIỆM VÀ KẾT QUẢ 15
4.1. Môi trường thực nghiệm 15
4.2. Các siêu tham số 16
4.3. Tiêu chí đánh giá 16
4.4. Kết quả thực nghiệm 17
4.4.1. Bảng số liệu 17

4.4.2. Biểu đồ 18
CHƯƠNG 5: THẢO LUẬN VÀ PHÂN TÍCH LỖI 22
5.1. Phân tích kết quả 22
5.2. Phân tích lỗi 23
5.2.1. Các dạng lỗi thường gặp 23
5.2.2. Nguyên nhân và phân tích sâu 25
5.2.3. Đề xuất hướng khắc phục 25
3. Hạn chế 26
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 27
6.1. Kết luận 27
6.2. Hướng phát triển 27
TÀI LIỆU THAM KHẢO 28
PHỤ LỤC 29

LỜI CAM ĐOAN

Em xin cam đoan rằng đề tài này là kết quả nghiên cứu và thực hiện của cá nhân em dưới sự hướng dẫn của thầy Nguyễn Tuấn Linh. Các nội dung, kết quả và chương trình được trình bày trong báo cáo là do em tự tìm hiểu, xây dựng và thực hiện. Những tài liệu, thông tin tham khảo từ các nguồn khác đều đã được ghi rõ trong phần tài liệu tham khảo.

Em xin hoàn toàn chịu trỏch nhiệm về tớnh trung thực của nội dung trong bỏo cỏo này.

Sinh viên thực hiện

Lương Văn Học

DANH MỤC CÁC TỪ VIẾT TẮT

1) SVM: Support Vector Machine
2) KNN: K-Nearest Neighbors
3) CNN: Convolutional Neural Network
4) VR: Virtual Reality (Thực tế ảo)
5) AR: Augmented Reality (Thực tế tăng cường)
6) CPU: Central Processing Unit (Bộ xử lý trung tâm)
7) RAM: Random Access Memory
8) GPU: Graphics Processing Unit
9) .npy: Định dạng file dữ liệu của NumPy
10) OpenCV: Open Source Computer Vision Library
11) NumPy: Numerical Python
12) MediaPipe: Thư viện xử lý hình ảnh của Google
13) Pygame: Thư viện lập trình game Python
14) Scikit-learn: Thư viện học máy Python
15) Accuracy: Độ chính xác
16) Train: Tập huấn luyện
17) Test: Tập kiểm tra
18) Confusion Matrix: Ma trận nhầm lẫn
19) Kernel: Hàm nhân (trong SVM)
20) Gamma: Tham số trong SVM
21) C: Tham số điều chỉnh độ phạt trong SVM

DANH MỤC HÌNH ẢNH

Hình 4.1: Giao diện thu thập dữ liệu cử chỉ tay 16

Hình 4.2: Kết quả đánh giá mô hình trên tập Train/Test (SVM và KNN) 17

Hình 4.3: Ma trận nhầm lẫn của mô hình SVM 18

Hình 4.4: Ma trận nhầm lẫn của mô hình KNN 19

Hình 4.5: Biểu đồ so sánh độ chính xác giữa SVM và KNN 20

Hình 5.1: Nhầm lẫn giữa các cử chỉ gần giống nhau 23

Hình 5.2: Nhầm lẫn khi chuyển cử chỉ nhanh 24

Hình 5.3: Tay bị che khuất, ra khỏi kung hình 24

LỜI NÓI ĐẦU

Trong những năm gần đây, trí tuệ nhân tạo và các công nghệ xử lý ảnh ngày càng phát triển và được ứng dụng rộng rãi trong nhiều lĩnh vực của đời sống. Một trong những hướng nghiên cứu được quan tâm là nhận dạng cử chỉ tay, cho phép con người tương tác với máy tính một cách tự nhiên và trực quan hơn mà không cần sử dụng các thiết bị điều khiển truyền thống như bàn phím hay chuột.

Xuất phát từ ý tưởng đó, em thực hiện đề tài "Nhận dạng cử chỉ tay điều khiển trò rắn săn mồi". Mục tiêu của đề tài là xây dựng một hệ thống có khả năng nhận diện cử chỉ tay của người dùng thông qua webcam và sử dụng kết quả nhận dạng để điều khiển hướng di chuyển của nhân vật trong trò chơi rắn săn mồi. Qua đó giúp minh họa khả năng ứng dụng của trí tuệ nhân tạo và học máy trong việc xây dựng các hệ thống tương tác người – máy.

Trong quá trình thực hiện đề tài, em đã tìm hiểu các kiến thức liên quan đến xử lý ảnh, thu thập dữ liệu, huấn luyện mô hình học máy và xây dựng giao diện trò chơi. Tuy đã cố gắng hoàn thành đề tài một cách tốt nhất, nhưng do kiến thức và thời gian còn hạn chế nên báo cáo không tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp của thầy để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn thầy Nguyễn Tuấn Linh đã hướng dẫn và hỗ trợ em trong quá trình thực hiện đề tài này.

Thái Nguyên, ngày 16 tháng 3 năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài "Nhận dạng cử chỉ tay điều khiển trò rắn săn mồi", em đã nhận được sự hướng dẫn và giúp đỡ tận tình từ thầy Nguyễn Tuấn Linh.

Em xin gửi lời cảm ơn chân thành đến thầy Nguyễn Tuấn Linh, người đã trực tiếp hướng dẫn, định hướng và hỗ trợ em trong suốt quá trình thực hiện đề tài. Những ý kiến góp ý và sự hướng dẫn của thầy đã giúp em hoàn thành bài tập lớn này.

Trong quá trình thực hiện đề tài, mặc dù em đã cố gắng tìm hiểu và hoàn thiện nội dung một cách tốt nhất, tuy nhiên bài làm vẫn có thể còn những thiếu sót. Em rất mong nhận được những ý kiến đóng góp và nhận xét của thầy để em có thể rút kinh nghiệm và hoàn thiện hơn trong quá trình học tập và nghiên cứu sau này.

Em xin chân thành cảm ơn.

CHƯƠNG 1. TỔNG QUAN

1.1 Đặt vấn đề

Trong những năm gần đây, các hệ thống tương tác giữa con người và máy tính ngày càng được quan tâm nghiên cứu. Thay vì sử dụng các thiết bị điều khiển truyền thống như bàn phím hay chuột, người dùng có thể tương tác với máy tính thông qua các cử chỉ tự nhiên của cơ thể. Nhận dạng cử chỉ tay là một trong những phương pháp phổ biến giúp tăng tính trực quan và tự nhiên trong quá trình tương tác giữa con người và máy tính.

Xuất phát từ ý tưởng đó, đề tài "Nhận dạng cử chỉ tay điều khiển trò rắn săn mồi" được thực hiện nhằm xây dựng một hệ thống có khả năng nhận diện cử chỉ tay của người dùng thông qua webcam và chuyển đổi các cử chỉ này thành lệnh điều khiển trong trò chơi rắn săn mồi. Thay vì sử dụng bàn phím để điều khiển hướng di chuyển của rắn, người chơi có thể sử dụng các cử chỉ tay tương ứng với các hướng như lên, xuống, trái và phải.

Hệ thống sử dụng camera để thu nhận hình ảnh bàn tay của người dùng, sau đó áp dụng các kỹ thuật xử lý ảnh và mô hình học máy để nhận diện cử chỉ tay. Kết quả nhận dạng sẽ được sử dụng để điều khiển hướng di chuyển của nhân vật rắn trong trò chơi theo thời gian thực.

Input:

- Hình ảnh bàn tay của người dùng được thu nhận từ webcam.
- Dữ liệu cử chỉ tay đã được thu thập và sử dụng để huấn luyện mô hình.

Output:

- Kết quả nhận dạng cử chỉ tay của người dùng (lên, xuống, trái, phải).
- Hướng di chuyển tương ứng của rắn trong trò chơi.

1.2 Lý do chọn đề tài

Trong những năm gần đây, trí tuệ nhân tạo và các công nghệ xử lý ảnh ngày càng được ứng dụng rộng rãi trong nhiều lĩnh vực của đời sống. Một trong những hướng phát triển quan trọng là xây dựng các hệ thống giúp con người có thể tương tác với máy tính một cách tự nhiên và trực quan hơn.

Nhận dạng cử chỉ tay là một trong những phương pháp phổ biến cho phép người dùng điều khiển và tương tác với máy tính mà không cần sử dụng các thiết bị điều khiển truyền thống như bàn phím hay chuột.

Việc nghiên cứu và xây dựng hệ thống nhận dạng cử chỉ tay không chỉ giúp minh họa cho khả năng ứng dụng của trí tuệ nhân tạo trong thực tế mà còn góp phần phát triển các phương thức tương tác mới giữa con người và máy tính. Công nghệ này có thể được ứng dụng trong nhiều lĩnh vực như điều khiển thiết bị thông minh, trò chơi tương tác, thực tế ảo (VR), thực tế tăng cường (AR) và các hệ thống điều khiển không chạm.

Xuất phát từ những lý do trên, đề tài "Nhận dạng cử chỉ tay điều khiển trò rắn săn mồi" được thực hiện nhằm nghiên cứu và xây dựng một hệ thống nhận diện cử chỉ tay thông qua webcam, từ đó sử dụng kết quả nhận dạng để điều khiển trò chơi. Đề tài giúp người thực hiện hiểu rõ hơn về quá trình thu thập dữ liệu, huấn luyện mô hình học máy và ứng dụng trí tuệ nhân tạo vào các hệ thống tương tác trong thực tế.

1.3. Mục tiêu của đề tài

- Xây dựng hệ thống nhận dạng cử chỉ tay thông qua webcam.
- Thu thập và xây dựng tập dữ liệu cho các cử chỉ tay gồm: lên, xuống, trái, phải.
- Áp dụng thuật toán học máy SVM để huấn luyện mô hình nhận dạng cử chỉ tay.
- Xây dựng trò chơi rắn săn mồi và kết nối với hệ thống nhận dạng cử chỉ tay để điều khiển hướng di chuyển của rắn.
- Đảm bảo hệ thống có khả năng nhận diện cử chỉ tay theo thời gian thực với độ chính xác cao.

1.4. Đóng góp của báo cáo

- Xây dựng hệ thống thu thập dữ liệu cử chỉ tay thông qua webcam.
- Xây dựng tập dữ liệu cho các cử chỉ tay gồm: lên, xuống, trái, phải.
- Áp dụng thuật toán SVM để huấn luyện mô hình nhận dạng cử chỉ tay.

- Xây dựng trò chơi rắn săn mồi và tích hợp với hệ thống nhận dạng cử chỉ tay.
- Thực hiện điều khiển trò chơi bằng cử chỉ tay theo thời gian thực.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT & CÁC NGHIÊN CỨU LIÊN QUAN

2.1. Các nghiên cứu liên quan

Trong những năm gần đây, bài toán nhận dạng cử chỉ tay đã được nhiều nhà nghiên cứu quan tâm và ứng dụng trong các hệ thống tương tác giữa con người và máy tính. Nhiều phương pháp khác nhau đã được đề xuất để nhận diện cử chỉ tay từ hình ảnh hoặc video thu thập từ camera.

Một số nghiên cứu sử dụng các kỹ thuật xử lý ảnh để phát hiện và trích xuất đặc trưng của bàn tay, sau đó áp dụng các thuật toán học máy như K-Nearest Neighbors (KNN), Support Vector Machine (SVM) hoặc Decision Tree để phân loại các cử chỉ tay. Những phương pháp này thường có ưu điểm là dễ triển khai và phù hợp với các hệ thống nhận dạng đơn giản.

Bên cạnh đó, nhiều nghiên cứu hiện nay sử dụng các mô hình học sâu như Convolutional Neural Network (CNN) để nhận dạng cử chỉ tay trực tiếp từ hình ảnh. Các mô hình này có khả năng học đặc trưng tự động từ dữ liệu và thường đạt độ chính xác cao hơn, tuy nhiên yêu cầu dữ liệu lớn và tài nguyên tính toán cao.

Ngoài ra, một số thư viện hỗ trợ như MediaPipe đã được phát triển để phát hiện và theo dõi các điểm đặc trưng của bàn tay trong thời gian thực. Các hệ thống nhận dạng cử chỉ tay có thể sử dụng các điểm đặc trưng này làm dữ liệu đầu vào cho các mô hình học máy nhằm tăng độ chính xác và tốc độ xử lý.

Trong đề tài này, hệ thống sử dụng dữ liệu cử chỉ tay thu thập từ webcam và áp dụng thuật toán SVM để xây dựng mô hình nhận dạng cử chỉ tay, sau đó kết nối với trò chơi rắn săn mồi để thực hiện điều khiển theo thời gian thực.

2.2. Cơ sở lý thuyết

Trong đề tài này, thuật toán học máy **Support Vector Machine (SVM)** được sử dụng để phân loại các cử chỉ tay dựa trên dữ liệu đã thu thập. SVM là

một thuật toán học có giám sát thường được sử dụng trong các bài toán phân loại dữ liệu.

Ý tưởng chính của SVM là tìm một siêu phẳng (hyperplane) trong không gian đặc trưng để phân tách các lớp dữ liệu khác nhau. Siêu phẳng này được lựa chọn sao cho khoảng cách từ siêu phẳng đến các điểm dữ liệu gần nhất của mỗi lớp là lớn nhất. Những điểm dữ liệu gần nhất này được gọi là các vector hỗ trợ (support vectors).

Giả sử tập dữ liệu huấn luyện gồm các cặp dữ liệu (xi,yi)(x_i, y_i)(xi,yi), trong đó xix_ixi là vector đặc trưng của dữ liệu đầu vào và yiy_iyi là nhãn lớp tương ứng. SVM tìm siêu phẳng có dạng:

w·x+b=0

Trong đó:

- w là vector trọng số.
- b là hệ số dịch chuyển.
- x là vector đặc trưng của dữ liệu.

Mục tiêu của SVM là tìm www và bbb sao cho khoảng cách giữa hai lớp dữ liệu là lớn nhất. Điều này tương đương với việc giải bài toán tối ưu:

$$\min \frac{1}{2}||w||^2$$

với điều kiện:

yi(w·xi+b)≥1

Sau khi mô hình được huấn luyện, với một dữ liệu mới xxx, mô hình sẽ dự đoán lớp của dữ liệu dựa trên dấu của hàm quyết định:

f(x)=w·x+b

Nếu f(x)>0 thì dữ liệu thuộc một lớp, nếu f(x)<0 thì thuộc lớp còn lại. Trong đề tài này, SVM được sử dụng để phân loại các cử chỉ tay thành các nhóm tương ứng với các hướng điều khiển như lên, xuống, trái và phải. Kết quả dự đoán sẽ được sử dụng để điều khiển hướng di chuyển của rắn trong trò chơi.

CHƯƠNG 3. PHƯƠNG PHÁP THỰC HIỆN

3.1. Mô tả dữ liệu

Dữ liệu sử dụng trong đề tài được thu thập trực tiếp thông qua webcam của máy tính. Người thực hiện tiến hành ghi nhận các cử chỉ tay tương ứng với bốn hướng điều khiển gồm: lên, xuống, trái và phải. Quá trình thu thập được thực hiện trong nhiều lần để đảm bảo tính đa dạng của dữ liệu.

Mỗi mẫu dữ liệu được xử lý và lưu trữ dưới dạng vector đặc trưng. Cụ thể, hệ thống trích xuất các điểm đặc trưng của bàn tay và biểu diễn mỗi mẫu bằng 42 giá trị tương ứng với tọa độ x và y của 21 điểm trên bàn tay. Dữ liệu được lưu dưới dạng các tệp `.npy` để thuận tiện cho việc xử lý và huấn luyện mô hình.

Tập dữ liệu bao gồm 4 lớp tương ứng với 4 cử chỉ tay. Mỗi lớp được thu thập khoảng 200–300 mẫu, tổng số mẫu của toàn bộ tập dữ liệu vào khoảng 800–1200 mẫu.

Sau khi thu thập, dữ liệu được chia thành hai phần gồm tập huấn luyện và tập kiểm tra. Trong đó, khoảng 80% dữ liệu được sử dụng để huấn luyện mô hình và 20% còn lại được sử dụng để kiểm tra và đánh giá hiệu quả của mô hình. Việc phân chia dữ liệu như vậy giúp đảm bảo mô hình có khả năng tổng quát tốt và hạn chế hiện tượng học thuộc dữ liệu.

3.2. Khám phá và Tiền xử lý dữ liệu

Sau khi thu thập, dữ liệu được tiến hành kiểm tra và phân tích nhằm đảm bảo chất lượng trước khi đưa vào huấn luyện mô hình. Quá trình khám phá dữ liệu chủ yếu tập trung vào việc kiểm tra số lượng mẫu của từng lớp cử chỉ (lên, xuống, trái, phải) để đảm bảo sự phân bố tương đối đồng đều. Ngoài ra, một số mẫu dữ liệu được quan sát trực tiếp để đánh giá tính hợp lệ của các giá trị tọa độ.

Trong bước làm sạch dữ liệu, các mẫu không hợp lệ sẽ được loại bỏ, chẳng hạn như các trường hợp không phát hiện được bàn tay hoặc dữ liệu bị

thiếu điểm đặc trưng. Việc loại bỏ các dữ liệu nhiễu giúp nâng cao độ chính xác của mô hình trong quá trình huấn luyện.

Tiếp theo, dữ liệu được chuẩn hóa nhằm đưa các giá trị về cùng một khoảng, giúp mô hình học máy hoạt động ổn định và hiệu quả hơn. Các nhãn cử chỉ tay được mã hóa thành dạng số để phù hợp với quá trình xử lý của mô hình. Cuối cùng, đặc trưng đầu vào được sử dụng trực tiếp là các tọa độ của bàn tay, đóng vai trò là vector đặc trưng cho mô hình SVM trong quá trình phân loại.

3.3. Mô hình đề xuất

Trong đề tài này, hệ thống sử dụng thuật toán học máy SVM (Support Vector Machine) để phân loại các cử chỉ tay. Đây là phương pháp học máy cổ điển, phù hợp với bài toán có số lượng dữ liệu không quá lớn và đặc trưng đầu vào đã được trích xuất rõ ràng. Dữ liệu đầu vào của mô hình là vector đặc trưng gồm các tọa độ điểm của bàn tay, và đầu ra là nhãn tương ứng với các cử chỉ: lên, xuống, trái, phải.

Quy trình hoạt động của hệ thống có thể được mô tả qua sơ đồ khối như sau:

Thu thập dữ liệu (webcam) → Xử lý ảnh → Trích xuất đặc trưng → Huấn luyện mô hình SVM → Dự đoán cử chỉ → Điều khiển trò chơi

Trong đó, dữ liệu từ webcam được xử lý để lấy thông tin đặc trưng của bàn tay, sau đó được đưa vào mô hình SVM để phân loại. Kết quả phân loại sẽ được sử dụng để điều khiển hướng di chuyển của rắn trong trò chơi.

Đối với thuật toán SVM, mục tiêu là tìm siêu phẳng phân tách các lớp dữ liệu sao cho khoảng cách giữa các lớp là lớn nhất. Hàm mất mát của SVM là hàm hinge loss, được sử dụng để tối ưu hóa việc phân loại chính xác các mẫu dữ liệu. Quá trình tối ưu được thực hiện thông qua việc giải bài toán tối ưu lồi nhằm tìm ra các tham số tốt nhất cho mô hình.

Do sử dụng phương pháp học máy cổ điển, hệ thống không cần sử dụng các thuật toán tối ưu phức tạp như trong học sâu, nhưng vẫn đảm bảo hiệu quả và tốc độ xử lý phù hợp cho bài toán nhận dạng cử chỉ tay theo thời gian thực.

CHƯƠNG 4: THỰC NGHIỆM VÀ KẾT QUẢ

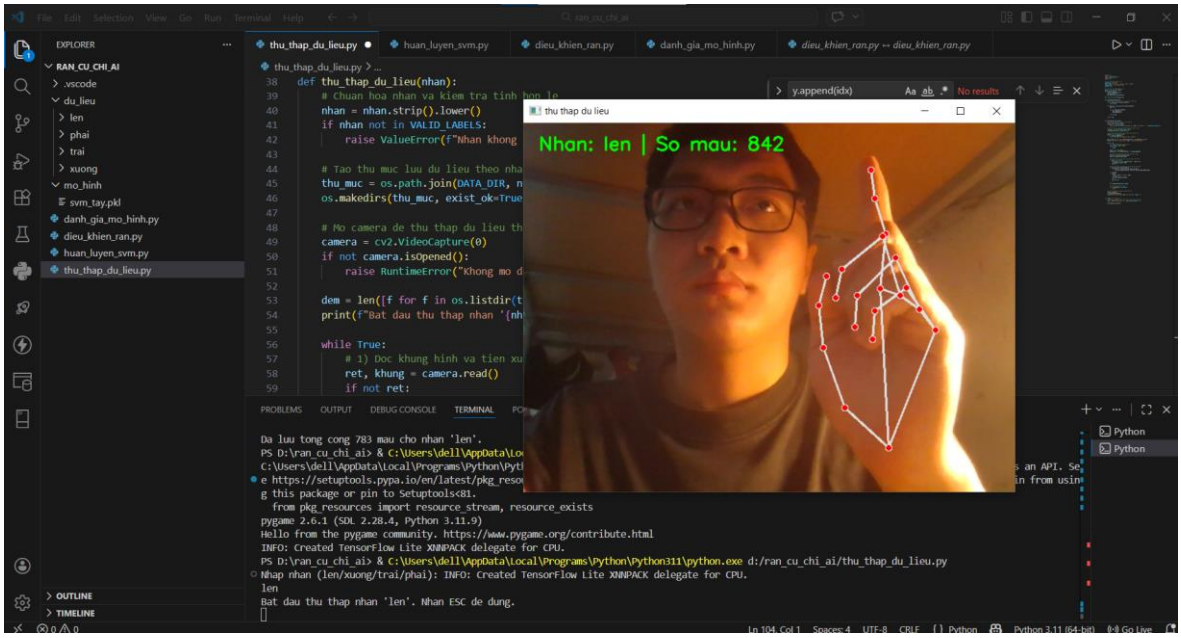
4.1. Môi trường thực nghiệm

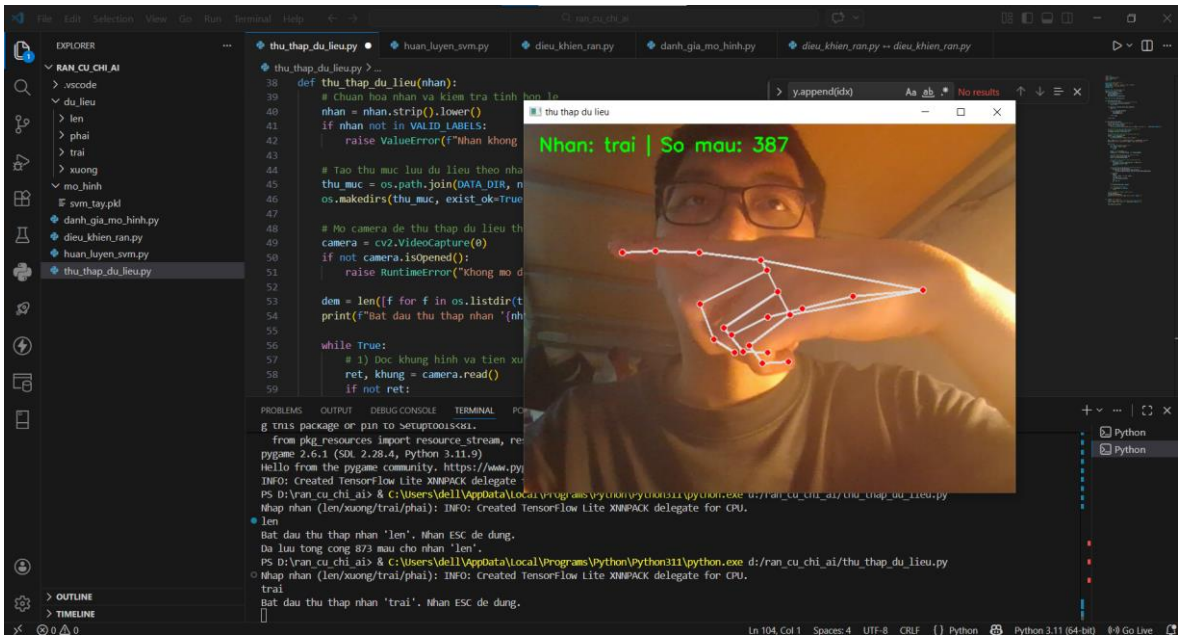
Hệ thống được triển khai và thử nghiệm trên máy tính cá nhân với cấu hình như sau:

- CPU: Intel Core i7
- RAM: 16GB
- GPU: Không sử dụng GPU rời
- Webcam: Webcam tích hợp

Các thư viện chính được sử dụng trong đề tài bao gồm:

- OpenCV: xử lý ảnh và truy cập webcam
- MediaPipe: phát hiện bàn tay và hỗ trợ thu thập dữ liệu
- NumPy: xử lý dữ liệu số
- Scikit-learn: xây dựng và huấn luyện mô hình SVM
- Pygame: xây dựng trò chơi rắn săn mồi





Hình 4.1: Giao diện thu thập dữ liệu cử chỉ tay

4.2. Các siêu tham số

Do sử dụng mô hình SVM – một thuật toán học máy cổ điển – nên hệ thống không sử dụng các siêu tham số như learning rate, batch size hay số epochs như trong học sâu. Thay vào đó, một số siêu tham số chính của SVM được sử dụng bao gồm:

- Kernel: sử dụng kernel tuyến tính (linear) để phân loại dữ liệu.
- Tham số C: điều chỉnh mức độ phạt khi phân loại sai, ảnh hưởng đến độ chính xác và khả năng tổng quát của mô hình.
- Gamma: không sử dụng do chọn kernel tuyến tính.

Các siêu tham số này được lựa chọn dựa trên thực nghiệm để đảm bảo mô hình hoạt động ổn định và đạt độ chính xác tốt.

4.3. Tiêu chí đánh giá

Để đánh giá hiệu quả của mô hình, đề tài sử dụng độ chính xác (Accuracy) làm tiêu chí chính. Accuracy đo lường tỷ lệ dự đoán đúng trên tổng số mẫu dữ liệu kiểm tra, phù hợp với bài toán phân loại nhiều lớp như nhận dạng cử chỉ tay.

Công thức tính Accuracy:

Accuracy = (Số mẫu dự đoán đúng) / (Tổng số mẫu)

Việc lựa chọn Accuracy là phù hợp vì dữ liệu giữa các lớp được thu thập tương đối đồng đều. Ngoài ra, tiêu chí này dễ hiểu và phản ánh trực quan hiệu quả của mô hình trong việc nhận diện đúng các cử chỉ tay.

4.4. Kết quả thực nghiệm

Trong phần này, đề tài tiến hành đánh giá hiệu quả của mô hình SVM và đồng thời sử dụng mô hình KNN như một phương pháp so sánh (baseline) nhằm làm rõ ưu điểm của mô hình đề xuất.

4.4.1. Bảng số liệu

Kết quả đánh giá mô hình trên tập dữ liệu được trình bày trong bảng sau:

Mô hình	Accuracy Train	Accuracy Test
SVM	99.49%	99.49%
KNN	99.68%	99.23%

Bảng 4.1: So sánh Accuracy (Train/Test) giữa mô hình SVM và KNN

Từ bảng số liệu có thể nhận thấy:

- Cả hai mô hình đều đạt độ chính xác rất cao
- Dữ liệu có tính phân tách tốt
- Mô hình SVM cho kết quả tốt hơn KNN

```
ran_cu_chi_ai/danh_gia_mo_hinh.py
Tong so mau: 1943
So mau train: 1554 | So mau test: 389

--- SVM ---
Train accuracy: 0.9949
Test accuracy:  0.9949

--- KNN ---
Train accuracy: 0.9968
Test accuracy:  0.9923
```

Hình 4.2: Kết quả đánh giá mô hình trên tập Train/Test (SVM và KNN)

4.4.2. Biểu đồ

Trong quá trình đánh giá mô hình, để trực quan hóa kết quả và phân tích sâu hơn về hiệu năng của từng thuật toán, đề tài sử dụng các biểu đồ sau:

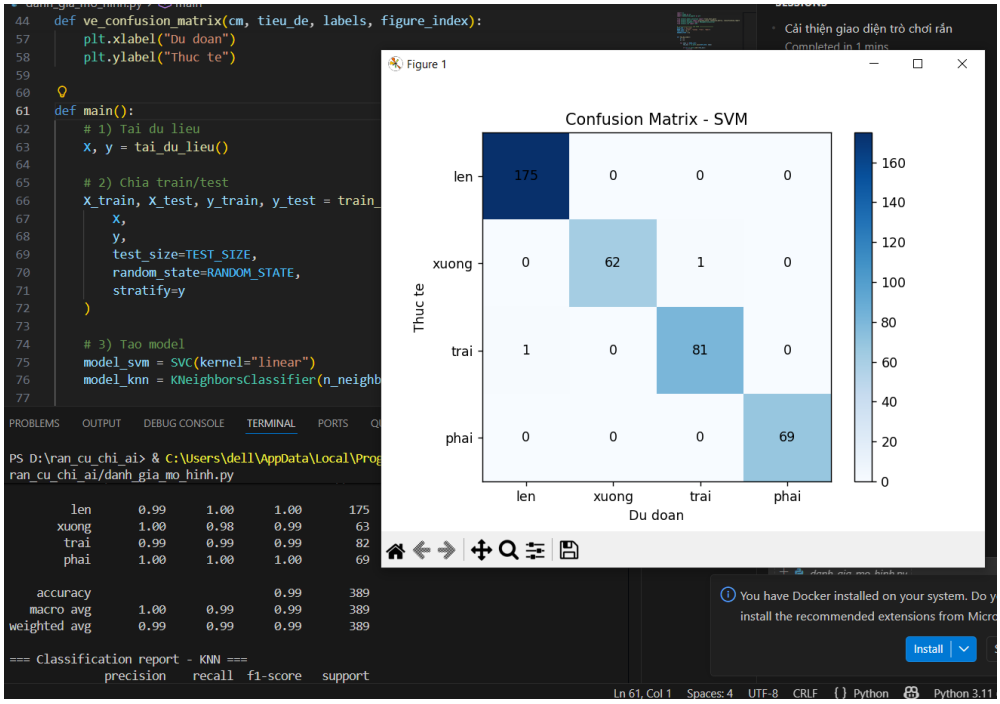
a) Ma trận nhầm lẫn (Confusion Matrix)

Ma trận nhầm lẫn được sử dụng nhằm đánh giá chi tiết khả năng phân loại của mô hình trên từng lớp dữ liệu cụ thể. Thông qua ma trận này, có thể quan sát được số lượng mẫu được dự đoán đúng và các trường hợp dự đoán sai giữa các lớp.

- Trục tung (Actual): thể hiện nhãn thực tế
- Trục hoành (Predicted): thể hiện nhãn dự đoán
- Các phần tử trên đường chéo chính: số lượng dự đoán đúng
- Các phần tử ngoài đường chéo: số lượng dự đoán sai

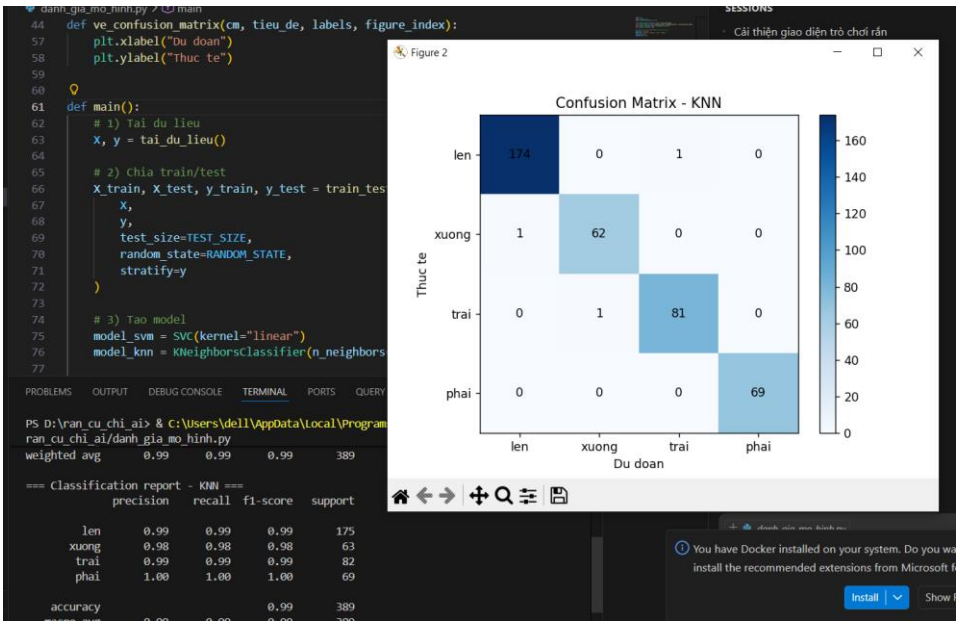
Trong đề tài, ma trận nhầm lẫn được xây dựng cho cả hai mô hình:

- **Mô hình SVM**



Hình 4.3: Ma trận nhầm lẫn của mô hình SVM

- **Mô hình KNN**



Hình 4.4: Ma trận nhầm lẫn của mô hình KNN

Từ các ma trận này có thể nhận thấy rằng phần lớn các mẫu được phân loại chính xác (nằm trên đường chéo chính), cho thấy mô hình hoạt động hiệu quả. Một số ít trường hợp bị nhầm lẫn giữa các lớp có đặc trưng tương tự nhau.

- Nhận xét:

Cả hai mô hình SVM và KNN đều cho kết quả phân loại rất tốt trên bài toán nhận dạng 4 cử chỉ (lên, xuống, trái, phải), thể hiện qua việc các giá trị trong ma trận nhầm lẫn tập trung chủ yếu trên đường chéo chính. Tuy nhiên, SVM cho hiệu năng nhỉnh hơn KNN: với tổng 389 mẫu kiểm tra, SVM chỉ dự đoán sai 2 mẫu (độ chính xác xấp xỉ 99,49%), trong khi KNN dự đoán sai 3 mẫu (độ chính xác xấp xỉ 99,23%).

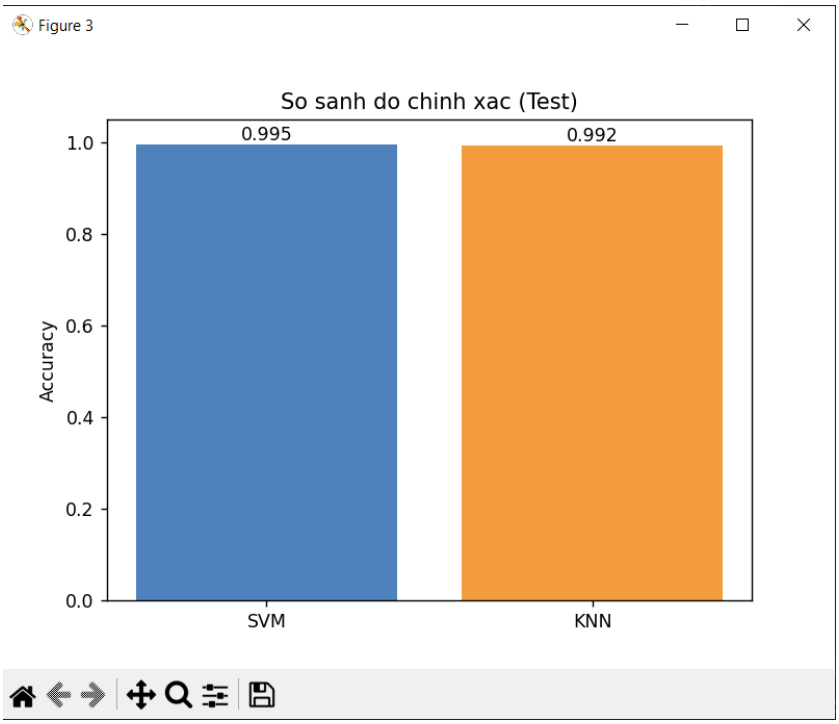
Xét theo từng lớp, cả hai mô hình đều nhận dạng lớp "phải" ổn định nhất (69/69 mẫu đúng, không có nhầm lẫn). Các lỗi còn lại chủ yếu xuất hiện ở những lớp có đặc trưng gần nhau, ví dụ SVM nhầm rất ít giữa "xuống" và "trái", còn KNN có xu hướng nhầm lẫn rải rác hơn giữa các cặp "lên–trái", "xuống–lên" và "trái–xuống". Nhìn chung, kết quả cho thấy SVM có khả năng tổng quát tốt hơn và ít nhầm lẫn hơn KNN trong thiết lập thí nghiệm

này, do đó phù hợp để lựa chọn làm mô hình điều khiển trò chơi theo thời gian thực.

b) Biểu đồ so sánh độ chính xác (Accuracy)

Để so sánh tổng quan hiệu năng giữa hai mô hình, đề tài sử dụng biểu đồ cột thể hiện độ chính xác (Accuracy) trên tập kiểm thử.

- Trục hoành: các mô hình (SVM, KNN)
- Trục tung: giá trị Accuracy
- Mỗi cột biểu diễn độ chính xác của một mô hình



Hình 4.5: Biểu đồ so sánh độ chính xác giữa SVM và KNN

Biểu đồ cho thấy:

- Cả hai mô hình đều đạt độ chính xác cao
- Mô hình SVM có độ chính xác nhỉnh hơn so với KNN
- Sự chênh lệch không lớn nhưng đủ để thấy SVM ổn định hơn

Nhận xét:

Biểu đồ cột "So sánh độ chính xác (Test)" thể hiện độ chính xác của hai mô hình trên tập kiểm tra. Kết quả cho thấy cả SVM và KNN đều đạt độ chính

xác rất cao (đều trên 0,99), chứng tỏ bộ dữ liệu và đặc trưng sử dụng có khả năng phân biệt các lớp cử chỉ tốt.

Cụ thể, SVM đạt Accuracy = 0,995, cao hơn KNN với Accuracy = 0,992. Chênh lệch 0,003 (tương đương 0,3%) tuy không lớn nhưng cho thấy SVM có xu hướng ổn định và tổng quát tốt hơn trong bài toán này. Vì vậy, xét theo tiêu chí độ chính xác trên tập test, SVM là lựa chọn phù hợp hơn để triển khai nhận dạng cử chỉ tay điều khiển trò chơi trong thời gian thực.

CHƯƠNG 5: THẢO LUẬN VÀ PHÂN TÍCH LỖI

5.1. Phân tích kết quả

Kết quả ở Chương 4 cho thấy mô hình SVM kernel tuyến tính đạt độ chính xác rất cao trên tập kiểm tra (xấp xỉ 99.49%) và ma trận nhầm lẫn tập trung chủ yếu trên đường chéo chính. Điều này chứng tỏ mô hình có khả năng học được ranh giới giữa các lớp cử chỉ (*lên, xuống, trái, phải*) từ dữ liệu đặc trưng đã thu thập. Các nguyên nhân chính giúp mô hình hoạt động tốt có thể phân tích như sau:

Thứ nhất, đặc trưng đầu vào mang tính mô tả tốt cho tư thế bàn tay. Đề tài sử dụng landmarks bàn tay từ MediaPipe và chuyển thành vector đặc trưng (tập tin .npy). Landmarks thể hiện cấu trúc hình học của bàn tay theo từng khung hình, do đó các cử chỉ điều hướng (lên/xuống/trái/phải) thường tạo ra các "mẫu hình" khác nhau trong không gian đặc trưng. Khi đặc trưng tốt, mô hình học máy cổ điển như SVM vẫn có thể đạt hiệu suất cao mà không cần mạng sâu.

Thứ hai, SVM tuyến tính phù hợp với bài toán và dữ liệu hiện có. Với dữ liệu landmarks, trong nhiều trường hợp ranh giới giữa các lớp có thể gần tuyến tính, đặc biệt nếu dữ liệu được thu thập tương đối "gọn" (một người dùng, môi trường tương đối ổn định, các cử chỉ được thực hiện rõ ràng). SVM tuyến tính có ưu điểm là tốc độ huấn luyện và dự đoán nhanh, giúp hệ thống dễ tích hợp vào trò chơi và đáp ứng yêu cầu thời gian thực. So với các mô hình phức tạp hơn, SVM tuyến tính còn ít tham số và dễ kiểm soát, phù hợp với phạm vi bài tập lớn.

Thứ ba, dữ liệu giữa các lớp tương đối cân bằng nên Accuracy phản ánh khá đúng hiệu năng mô hình. Trong trường hợp dữ liệu bị lệch lớp mạnh, Accuracy có thể gây hiểu nhầm; tuy nhiên trong đề tài, số lượng mẫu cho mỗi cử chỉ được thu thập tương đối đồng đều nên Accuracy là lựa chọn hợp lý, đồng thời dễ trình bày và so sánh với mô hình baseline (KNN).

Tuy vậy, cần lưu ý rằng độ chính xác cao trên tập test chưa đảm bảo mô hình luôn ổn định trong mọi tình huống thực tế. Khi triển khai thời gian thực, hệ thống còn phụ thuộc vào chất lượng phát hiện tay của MediaPipe,

điều kiện môi trường, khoảng cách và góc đặt tay. Vì vậy, ngoài kết quả định lượng trên tập test, việc quan sát hành vi mô hình trong demo thực tế (độ trễ, độ ổn định lệnh) cũng là yếu tố quan trọng để đánh giá mức độ "sẵn sàng ứng dụng" của hệ thống.

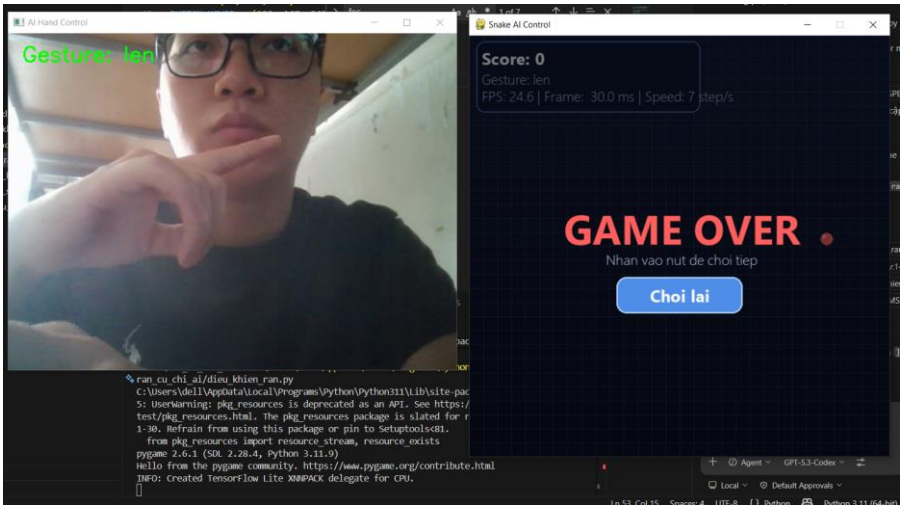
5.2. Phân tích lỗi

Mặc dù đạt kết quả tốt, mô hình vẫn có một số trường hợp dự đoán sai (thể hiện qua các ô ngoài đường chéo của ma trận nhầm lẫn). Với tập kiểm tra 389 mẫu, SVM chỉ sai 2 mẫu và KNN sai 3 mẫu, cho thấy lỗi không nhiều nhưng vẫn cần phân tích để hiểu nguyên nhân và định hướng cải thiện.

5.2.1. Các dạng lỗi thường gặp

1) Nhầm lẫn giữa các cử chỉ có đặc trưng gần nhau

Một số cử chỉ có thể bị nhầm khi tư thế tay không rõ ràng hoặc người dùng thực hiện không "chuẩn" theo mẫu thu thập. Ví dụ, khi tay nghiêng hoặc cổ tay xoay nhẹ, vector landmarks có thể trở nên giống với lớp khác. Đây là dạng lỗi phổ biến trong các bài toán phân loại dựa trên hình học, khi các lớp bị chồng lấn trong không gian đặc trưng.

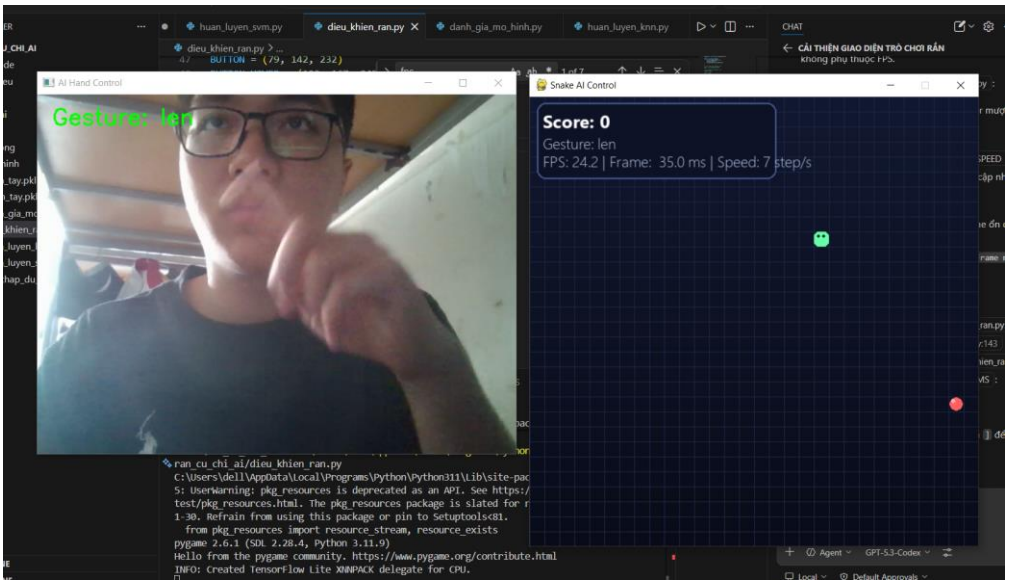


Hình 5.1: Nhầm lẫn giữa các cử chỉ gần giống nhau

2) Sai khi chuyển cử chỉ nhanh (dao động theo thời gian)

Trong môi trường real-time, người dùng thường đổi hướng liên tục. Khi đó, landmarks có thể thay đổi mạnh giữa các frame, hoặc MediaPipe trả về điểm mốc dao động nhẹ (jitter). Việc dự đoán từng frame độc lập có thể khiến nhãn dự đoán "nhảy" qua lại, gây ra cảm giác điều khiển không ổn định. Về

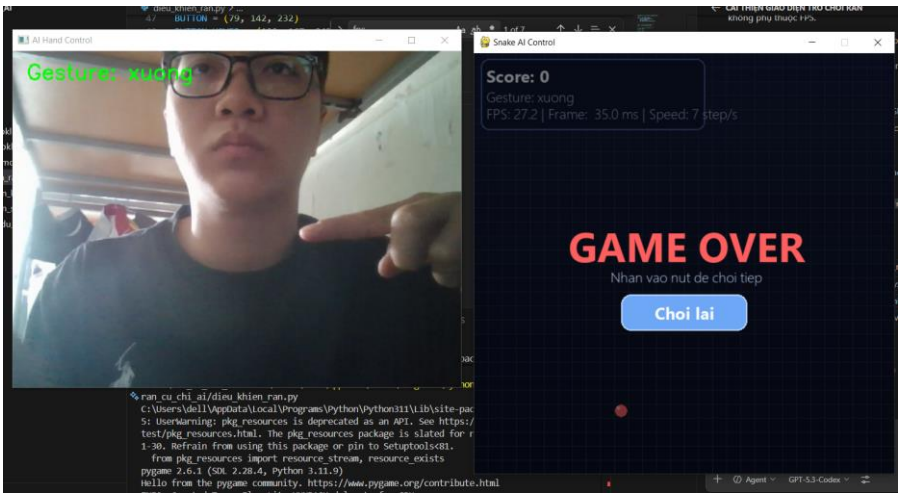
bản chất, đây là vấn đề nhiễu theo thời gian và có thể giảm bằng cách làm mượt dự đoán qua nhiều frame.



Hình 5.2: Nhầm lẫn khi chuyển cử chỉ nhanh

3) Không nhận diện hoặc nhận sai khi tay bị che khuất / ra khỏi khung hình / ánh sáng xấu

Khi bàn tay bị che một phần, ra sát biên khung hình, hoặc điều kiện ánh sáng yếu/ngược sáng, MediaPipe có thể phát hiện thiếu điểm hoặc điểm mốc bị lệch. Vì mô hình SVM/KNN phụ thuộc trực tiếp vào vector landmarks, nếu landmarks sai thì dự đoán sai là điều khó tránh. Đây là dạng lỗi xuất phát từ bước phát hiện (detection) trước khi phân loại, tức lỗi đến từ cả pipeline chứ không chỉ do classifier.



Hình 5.3: Tay bị che khuất, ra khỏi kung hình

5.2.2. Nguyên nhân và phân tích sâu

- Giới hạn dữ liệu thu thập: Tập dữ liệu có thể chưa bao quát đủ các biến thiên thực tế như nhiều người dùng, nhiều kích thước bàn tay, nhiều độ cao/thấp của tay trong khung hình, thay đổi ánh sáng, hoặc thay đổi nền. Khi gặp dữ liệu "lạ" so với lúc train, mô hình có thể giảm độ chính xác.
- Thiếu chuẩn hóa đặc trưng: Nếu vector landmarks dùng tọa độ tuyệt đối (x, y) trong khung hình, mô hình có thể nhạy với việc bàn tay nằm lệch trái/phải, gần/xa camera. Điều này làm tăng sai số khi người dùng không giữ tay ở đúng vị trí như lúc thu thập dữ liệu.
- Ranh giới tuyến tính có giới hạn: SVM tuyến tính nhanh và đơn giản, nhưng trong trường hợp dữ liệu chồng lấn mạnh hoặc bị nhiễu, ranh giới tuyến tính có thể không đủ linh hoạt. Khi đó, các kernel phi tuyến (RBF) hoặc mô hình khác có thể tách lớp tốt hơn.

5.2.3. Đề xuất hướng khắc phục

Để giảm lỗi và tăng tính ổn định khi chạy thực tế, đề tài có thể áp dụng một số cải tiến sau:

- Mở rộng dữ liệu: thu thập thêm mẫu ở nhiều điều kiện ánh sáng và nhiều góc tay; tăng số người tham gia để dữ liệu đa dạng hơn.
- Chuẩn hóa landmarks: đưa về hệ tọa độ tương đối (lấy cổ tay làm gốc), hoặc chuẩn hóa theo kích thước bàn tay (ví dụ theo khoảng cách giữa hai mốc xương ngón), giúp giảm phụ thuộc vị trí/độ lớn.
- Làm mượt dự đoán theo thời gian: dùng majority vote trong 5–10 frame, hoặc chỉ đổi hướng khi dự đoán trùng nhau N lần liên tiếp; điều này giảm "nhảy nhãn" khi tay rung/đổi cử chỉ nhanh.
- Thử mô hình mạnh hơn: ngoài SVM linear, có thể thử SVM RBF, Random Forest hoặc MLP để kiểm tra khả năng cải thiện trong các tình huống phức tạp.

3. Hạn chế

Bên cạnh những kết quả đạt được, hệ thống vẫn còn tồn tại một số hạn chế như sau:

- Tập dữ liệu còn nhỏ, chưa bao quát được nhiều trường hợp thực tế
- Phụ thuộc nhiều vào quá trình thu thập dữ liệu ban đầu
- Độ chính xác chưa ổn định trong các điều kiện môi trường khác nhau
- Chưa xử lý tốt các trường hợp nhiễu hoặc thay đổi ánh sáng
- Chỉ nhận diện được các cử chỉ đơn giản, chưa mở rộng sang các cử chỉ phức tạp hơn
- Chưa áp dụng các phương pháp nâng cao hoặc mô hình học sâu để cải thiện hiệu suất

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Sau quá trình nghiên cứu và thực hiện, đề tài đã xây dựng thành công một hệ thống nhận dạng cử chỉ tay sử dụng webcam kết hợp với mô hình học máy SVM. Hệ thống có khả năng thu thập dữ liệu, xử lý ảnh, trích xuất đặc trưng và phân loại các cử chỉ tay cơ bản gồm: lên, xuống, trái, phải. Kết quả nhận dạng được sử dụng để điều khiển trò chơi rắn săn mồi trong thời gian thực.

Kết quả thực nghiệm cho thấy mô hình hoạt động ổn định, đạt độ chính xác tương đối cao và có thể áp dụng trong thực tế với các điều kiện môi trường phù hợp. Hệ thống đã đáp ứng được các mục tiêu đề ra ban đầu như xây dựng mô hình nhận dạng, tích hợp vào ứng dụng thực tế và đảm bảo khả năng xử lý theo thời gian thực. Qua đó, đề tài đã thể hiện được tính ứng dụng của trí tuệ nhân tạo trong lĩnh vực tương tác người – máy.

6.2. Hướng phát triển

Trong tương lai, đề tài có thể được phát triển và hoàn thiện hơn theo nhiều hướng khác nhau. Trước hết, cần mở rộng tập dữ liệu cả về số lượng và sự đa dạng (nhiều người dùng, nhiều góc độ, điều kiện ánh sáng khác nhau) để nâng cao độ chính xác và khả năng tổng quát của mô hình.

Bên cạnh đó, có thể nghiên cứu và áp dụng các mô hình học sâu như CNN hoặc kết hợp nhiều mô hình để cải thiện hiệu suất nhận dạng, đặc biệt trong các trường hợp phức tạp. Việc tối ưu các siêu tham số của mô hình SVM cũng là một hướng giúp cải thiện kết quả.

Ngoài ra, hệ thống có thể được mở rộng để nhận diện nhiều loại cử chỉ hơn, từ đó ứng dụng vào các lĩnh vực thực tế như điều khiển thiết bị thông minh, hỗ trợ người khuyết tật, hoặc phát triển các trò chơi tương tác không chạm. Điều này sẽ giúp nâng cao tính ứng dụng và giá trị thực tiễn của đề tài.

TÀI LIỆU THAM KHẢO

1) Cortes, C., & Vapnik, V. (1995). Support-Vector Networks. Machine Learning, 20(3), 273–297.
2) Scikit-learn. Support Vector Machines. Truy cập tại: https://scikit-learn.org/stable/modules/svm.html
3) Google. MediaPipe Hands. Truy cập tại: https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
4) OpenCV. Open Source Computer Vision Library. Truy cập tại: https://opencv.org/
5) NumPy. NumPy Documentation. Truy cập tại: https://numpy.org/doc/
6) Pygame. Pygame Documentation. Truy cập tại: https://www.pygame.org/docs/
7) Nguyễn Thanh Thủy (2017). Giáo trình Trí tuệ nhân tạo. Nhà xuất bản Giáo dục Việt Nam.
8) Lê Hoài Bắc (2020). Nhập môn Học máy. Nhà xuất bản Đại học Quốc gia TP.HCM.
9) Nguyễn Đình Thắng (2018). Xử lý ảnh số. Nhà xuất bản Khoa học và Kỹ thuật.

PHỤ LỤC

Link Github: https://github.com/LuongHoc/BTL_TTNT-HM

QR Github:

Link Youtube demo: https://youtu.be/0FSxGwlc_Pw

QR Youtube: